

COMP7005 Week 1 Lab Report

Will Otterbein, A01372608

Results

Here are the results of executing the compiled pipe binary 10 times (just to confirm they are the same):

```
otter@W0surface:~/src/comp7005/labs/week1_a01372608$ gcc -std=c11 -Wall -Wextra -Wpedantic -g
otter@W0surface:~/src/comp7005/labs/week1_a01372608$ for run in {1..10}; do ./pipe; done
[Parent] Monitoring child PID 6317
[Parent] SIGUSR1/EINTR checkpoint
[Parent] Child exited normally with status 0
[Parent] PASS: iterations=20 sum=210 signal=SIGUSR1
[Parent] Monitoring child PID 6319
[Parent] SIGUSR1/EINTR checkpoint
[Parent] Child exited normally with status 0
[Parent] PASS: iterations=20 sum=210 signal=SIGUSR1
[Parent] Monitoring child PID 6321
[Parent] SIGUSR1/EINTR checkpoint
[Parent] Child exited normally with status 0
[Parent] PASS: iterations=20 sum=210 signal=SIGUSR1
[Parent] Monitoring child PID 6323
[Parent] SIGUSR1/EINTR checkpoint
[Parent] Child exited normally with status 0
[Parent] PASS: iterations=20 sum=210 signal=SIGUSR1
[Parent] Monitoring child PID 6327
[Parent] SIGUSR1/EINTR checkpoint
[Parent] Child exited normally with status 0
[Parent] PASS: iterations=20 sum=210 signal=SIGUSR1
[Parent] Monitoring child PID 6330
[Parent] SIGUSR1/EINTR checkpoint
[Parent] Child exited normally with status 0
[Parent] PASS: iterations=20 sum=210 signal=SIGUSR1
[Parent] Monitoring child PID 6332
[Parent] SIGUSR1/EINTR checkpoint
[Parent] Child exited normally with status 0
[Parent] PASS: iterations=20 sum=210 signal=SIGUSR1
[Parent] Monitoring child PID 6334
[Parent] SIGUSR1/EINTR checkpoint
[Parent] Child exited normally with status 0
[Parent] PASS: iterations=20 sum=210 signal=SIGUSR1
[Parent] Monitoring child PID 6336
[Parent] SIGUSR1/EINTR checkpoint
[Parent] Child exited normally with status 0
[Parent] PASS: iterations=20 sum=210 signal=SIGUSR1
[Parent] Monitoring child PID 6338
[Parent] SIGUSR1/EINTR checkpoint
[Parent] Child exited normally with status 0
[Parent] PASS: iterations=20 sum=210 signal=SIGUSR1
otter@W0surface:~/src/comp7005/labs/week1_a01372608$
```

TODO 1

parent_handle_ready implementation

```
static void parent_handle_ready(int signal_number)
{
    // TODO 1: Set parent_result_ready. Do not call print
    // parent_result_ready = 1;
    (void)signal_number;
    parent_result_ready = 1;
}
```

- In this snippet, simple we are setting the parent_result_ready flag to have the value of 1 (simply meaning this has been received), casting the signal number parameter to void is done to silence a compiler warning that was complaining about the unused parameter (Cole, classmate, told me about this solution). I had initially just used to signal number as the flag value, but I decided I liked Cole's solution better so that the flag value (and checks against the flag value), need not be dependent on the signals value.
- parent_result_ready as per the template code is a volatile variable of the sig_atomic_t type.

TODO 2

write_all implementation

```
static int write_all(int fd, const void *buffer, size_t length)
{
    // DONE ~TODO~ 2: Loop until write() accepts every byte. Retry when errno =
    = EINTR.
    const unsigned char *bytes = buffer;
    size_t off = 0;
    while ( off < length ) {
        ssize_t n = write(fd, bytes + off, length - off); // signed because result
        // may be negative
        if ( n > 0 ) {
            off += (size_t)n; // casting to unsigned
        } else if ( n == -1 && errno == EINTR ) {
            continue;
        } else {
            return -1;
        }
    }
    return 0;
}
```

- This function creates an unsigned char pointer bytes and an offset tracker of type size_t named off.
- the function loops on the condition that the value of off (which is a byte offset) is less than the length of the input buffer, variable length. For each iteration, the write syscall is attempted, starting from the bytes pointer plus whatever the offset is currently, and for a length that is the entire buffers length minus the offset (ie. the remaining memory segment). If write succeeds (n > 0) then we increment the offset & loop, if there

is an error ($n == -1$) but `errno` is `EINTR` we continue, otherwise the function returns w/ an error value of `-1`. Successful return value is `0`.

TODO 3

`read_exact` implementation

```
static int read_exact(int fd, void *buffer, size_t length)
{
    // DONE ~TODO~ 3: Loop until read() receives exactly length bytes.
    // Treat early EOF as an error and retry when errno == EINTR.
    unsigned char *bytes = buffer;
    size_t off = 0;
    while ( off < length ) {
        ssize_t n = read(fd, bytes + off, length - off); // signed because re
        sults may be negative
        if ( n > 0 ) {
            off += (size_t)n; // casting to unsigned
        } else if ( n == 0 ) { // when an early EOF is encountered
            return -1;
        } else if (errno != EINTR) {
            return -1;
        }
    }
    return 0;
}
```

- This function creates an unsigned char pointer `bytes` to the buffer and an offset tracker of type `size_t` named `off`.
- The function loops on the condition that the value of the offset tracker `off` is less than the length of the input buffer (named `buffer`). For each iteration the `read` syscall is attempted, starting from the `bytes` pointer plus whatever the offset is currently, and for a length that is the entire buffers length minus the offset (ie. the remaining memory segment). If the read succeeds ($n > 0$) then we increment the offset & loop (if the entire segment wasn't read then we will read the remainder), if there was an error executing read that does not have `errno` equalling `EINTR`, or EOF is encountered early, we return `-1` error value, and a successful return is once again `0`.

TODO 4

`child_work` result struct field population

```
static void child_work(int result_write_fd)
{
    struct pipe_result result = {0, 0};

    for (unsigned long iteration = 1; iteration ≤ WORK_ITERATIONS; iteration++)
    {
        // DONE ~TODO~ 4: Update result.iterations and add iteration to result.
        sum.
        result.iterations = iteration;
        result.sum += iteration;
        sleep_for_work_pacing();
    }
}
```

- this is the implementation of the child's work, which is tracking the current iteration number and then

incrementing the counter with the value of the iteration.

TODO 5

child_work result struct write to pipe, send signal to parent, close write file descriptor

```
static void child_work(int result_write_fd)
{
    struct pipe_result result = {0, 0};

    for (unsigned long iteration = 1; iteration ≤ WORK_ITERATIONS; iteration++)
    {
        // DONE ~TODO~ 4: Update result.iterations and add iteration to result.
        sum.
        result.iterations = iteration;
        result.sum += iteration;
        sleep_for_work_pacing();
    }

    // DONE ~TODO~ 5:
    // Publish the completed result to the parent, then notify the parent.
    // After the notification, release the pipe descriptor and exit normally.
    //
    // Requirements:
    // - The complete result must enter the pipe before SIGUSR1 is sent.
    // - Check every system-call/helper result.
    // - Use a nonzero child exit status if an operation fails.
    // - Close the child's write end before exiting.

    if (write_all(result_write_fd, &result, sizeof(result)) == -1) {
        perror("[Child] `write_all` call errored");
        _exit(127);
    }
    if (kill(getppid(), SIGUSR1) == -1) {
        perror("[Child] child process termination");
        _exit(127);
    }
    if (close(result_write_fd) == -1) {
        perror("[Child] child process failed to close write pipe");
        _exit(127);
    }

    // artificial delay to ensure that the handler in the loop works
    sleep_for_work_pacing();

    _exit(0);
}
```

- After the snippet from todo4 run populating the fields of the result struct (pipe_result_type), we invoke the write_all function previously described, we give the result_write_fd as the file descriptor (if this call fails returning -1, we log this error and exit w/ a non-zero exit code). Then the kill syscall is used to send the SIGUSR1 signal to the parent process. If this syscall fails, we log an error and exit w/ a non-zero exit code. The final syscall in this method is the close syscall on the result_write_fd file descriptor, checking for errors (logging + error exiting from the function if an error with this syscall occurs).
- We get the parent's PID using the getppid syscall (as per man 2 getppid there is no need to handle syscall errors for this syscall, because it always succeeds).

for this TODO, I added a sleep after closing the file descriptor & sending the signal to test that the

logic in the loop & the outer logic were both accomplishing the same thing, I found without this delay the child process would always exit before the signal was processed in the main loop.

TODO 6

```
if (pid == 0) {  
    // DONE ~TODO~ 6:  
    // The child writes the result. It does not read from this pipe.  
    // Close the unused read end and check close(). Keep the write end open.  
    if (close(result_pipe[0]) == -1) {  
        perror("[Child] failed to close the pipe read file descriptor");  
        _exit(127);  
    }  
    child_work(result_pipe[1]);  
}
```

- if the return value of `fork` is the value 0 this means that the execution context is that of the child process, and so this todo snippet in main is responsible for closing the `result_pipe` read file descriptor (index 0) of the pair of file descriptors returned by the pipe syscall earlier in the template code. The child process only needs to write data to the `result_pipe`.
- we also invoke `child_work` with the `result_pipe` write file descriptor (index 1 of the pair returned by the earlier pipe syscall).

TODO 7

```
// DONE ~TODO~ 7:  
// The parent reads the child's result. It does not write to this pipe.  
// Close the unused write end and check close(). Keep the read end open.  
if (close(result_pipe[1]) == -1) {  
    perror("[Parent] failed to close the pipe write file descriptor");  
    return 4;  
}
```

- proceeding with the parent process executing context in main, we are closing the write pair of the `result_pipe` file descriptors (index 1), as the parent process does not need to write any data to the pipe (`close` syscall).
- we handle the error state of the `close` syscall (checking if the return of this is -1).

TODO 8

```
struct pipe_result result = {0, 0};
int result_received = 0;
int notification_reported = 0;
int observed_child_exit = 0;
int status = 0;

for (;;) {
    errno = 0;
    const pid_t waited = waitpid(pid, &status, 0);

    if (waited == pid) {
        observed_child_exit = 1;
        break;
    }

    if (waited == -1 && errno == EINTR) {
        // DONE ~TODO~ 8: When parent_result_ready is
        // - read_exact() one struct pipe_result from re
        // - set result_received and notification_report
        // - clear parent_result_ready
        // - print the SIGUSR1/EINTR checkpoint
        //
        if (parent_result_ready == 1) {
            if (result_received == 0) {
                if (read_exact(result_pipe[0],
                               &result,
                               sizeof(result)) == -1) {
                    perror("[Parent] `read_exact` operation
                    return 5;
                }
                result_received = 1;
            }
            notification_reported = 1;
            parent_result_ready = 0;
            printf("[Parent] SIGUSR1/EINTR checkpoint\n");
        }
    }
}
```

```
        continue;
    }

    if (waited == -1) {
        perror("[Parent] waitpid");
        break;
    }
}
```

- This loop is where the parent process is waiting for the child process to signal. Herein the implementation is under the `if (waited == -1 && errno == EINTR)` condition, meaning that `waitpid` syscall is waiting still for the child process and that the syscall was interrupted by a signal. Then the program checks the flag `parent_result_ready` equals are expected signal of `SIGUSR1`, if this is true we also check that `result_received` is 0, then we execute our implementation of `read_exact` into the target struct named `result`, checking for the error return value of -1 (if there is an error, we log this and return from the program). If `read_exact` executes successfully, then we set `result_received` to 1.
- Regardless of whatever happens with `read`, we set `notification_reported` to 1 inside the `if` block (the one checking the signal) to record that the notification was reported (we write a checkpoint message via `printf` as well) and reset `parent_result_ready` to 0.

TODO 9

```
// DONE ~TODO~ 9:
// The child may signal and exit before the parent proces
// Then waitpid() can return the child PID, so the TODO 8
// After the loop, handle any unread result and unreported
if (result_received == 0) {
    if (read_exact(result_pipe[0],
        &result,
        sizeof(result)) == -1) {
        perror("[Parent] `read_exact` operation failed");
        return 6;
    }
    result_received = 1;
    printf("[Parent] handling unread result\n");
}

if (parent_result_ready == 1) {
    notification_reported = 1;
    parent_result_ready = 0;
}
```

- Finally we check both flags after the "main" loop, in the case that the child process exits before the signal is processed by the condition in the earlier loop. If `result_received` is still zero, then we execute the read operation (with error handling the same as before, an argument is to be made that I should have created a function for this as the logic is repeated), we set `result_received` to 1 indicating now that we have received the results from the pipe & wrote them into the `result` struct.
- The program also checks that the `parent_result_ready` flag is set signal, if so it sets `notification_reported` to 1 & clears the `parent_result_ready` flag (sets it to zero).